垃圾回收机制：

js的内存回收机制（垃圾回收机制）：一个函数在执行开始的时候,会给其中定义的变量划分内存空间保存,以备后面的语句所用,等到函数执行完毕返回(return)了,这些变量就被认为是无用的了，对应的空间也就被回收了。

下次再执行此函数的时候，所有的变量又回到最初的状态，重新赋值使用。

但是如果这个函数内部又嵌套了另一个函数(这就是闭包了)，而这个函数是有可能在外部被调用到的。

并且这个内部函数又使用了外部函数的某些变量的话.这种内存回收机制就会出现问题。

如果在外部函数返回后，又直接调用了内部函数，那么内部函数就无法读取到他所需要的外部函数中变量的值了。所以js解释器在遇到函数定义的时候会自动把函数和他可能使用的变量(包括本地变量和父级和祖先级函数的变量(自由变量))一起保存起来。

也就是构建一个闭包，这些变量将不会被内存回收器所回收，只有当内部的函数不可能被调用以后(例如被删除了,或者没有了指针)，才会销毁这个闭包,而没有任何一个闭包引用的变量才会被下一次内存回收启动时所回收。

什么是闭包？闭包形成的条件是什么？

1. 函数嵌套函数，
2. 内部函数使用了外部函数的变量（参数）；

闭包的优点与缺点:

1. 会保留需要存储的变量
2. 避免全局变量被污染;

闭包的问题:

1. 不关闭页面的时候，闭包会一直占用内存空间;
2. 无法回收变量，导致内存泄露/内存无法得到释放;

```html
<body>

    <button>1</button>
    <button>2</button>
    <button>3</button>
    <button>4</button>
    <button>5</button>
    <button>6</button>
    <button>7</button>
    <button>8</button>



    <script>

        var btns = document.getElementsByTagName('button');
        for( var i = 0;i<btns.length;i++ ){

            // btns[i].i = i;  //保存在自定义属性的;
            // btns[i].onclick = function(){
            //     // alert( i )
            //     console.log( this.i )
            // }

            (function(i){
                // console.log( i )

                //形成闭包：函数嵌套函数，内部函数使用外面函数的值;
                btns[i].onclick = function(){
                    console.log( i )
                }

            })(i)


        }



    </script>


</body>
</html>
```

递归

如果一个函数在内部可以调用其本身，那么这个函数就是递归函数

简单理解：函数内部自己调用自己这个函数就是递归函数